EXPERIMENT 08

Course Outcome: CO5 – Illustrate the concepts of various front-end web application developments, mockApi technologies frameworks using different web development tools.

KEHAN DESAI A2 16010125050

Simple Single Page Web Application using React JS

Aim: To develop a simple Single Page Web Application using React JS using reusable components, JavaScript data, JSX and React Router.

Objectives

- Understand the concept of a Single Page Application (SPA).
- Create multiple React components representing application views.
- Implement client-side navigation using React Router.
- Use JavaScript arrays and map() to generate UI elements.
- Reuse JSX, components and state concepts learned in Experiment 07.

NOTE: Refer to the "Case Study you selected" and complete the tasks in React. This write-up uses Student Information Application as a sample.

Software Requirements

- Node.js and npm
- Visual Studio Code
- Google Chrome or another modern browser

Theory

Single Page Application (SPA)

An SPA loads an application shell and changes the displayed view on the client without requiring a complete browser document reload for every internal route. React can implement this model using a routing library.

React Router

React Router provides BrowserRouter, Routes, Route and Link for mapping URL paths to React components and navigating between views.

Link and HTML Navigation

- HTML commonly uses About.
- React Router uses <Link to="/about">About</Link> for client-side navigation.
- The goal is to switch application views without a full page navigation.

JavaScript map() in React

```
students.map(student => (
  <p key={student.id}>{student.name} - {student.course}</p>
));
```

Problem Statement

Develop a simple Student Portal SPA containing Home, Students and About views. Navigation must use React Router. The Students view must generate records from a JavaScript array using map().

Tasks to be Given to Students

1. Create a React application named student-portal.
2. Install react-router-dom.
3. Create Home.js, Students.js and About.js components.
4. Create navigation links for Home, Students and About.
5. Configure BrowserRouter, Routes and Route.
6. Display a welcome message on Home.
7. Display at least three student records using an array and map().
8. Use a unique key for each generated item.
9. Display application information on About.
10. Apply simple CSS.
11. Test all routes and capture screenshots.

Procedure

12. Open Command Prompt and navigate to the practical folder.
13. Run: npx create-react-app student-portal
14. Run: cd student-portal
15. Run: npm install react-router-dom
16. Run: npm start
17. Create a pages folder inside src.
18. Create Home.js, Students.js and About.js.
19. Write the components.
20. Import them into App.js.
21. Configure BrowserRouter, Routes, Route and Link.
22. Test /, /students and /about.

Program – Home.js

```
function Home() {
  return (
    <div>
      <h1>Welcome to Student Portal</h1>
      <p>This is the home page.</p>
```

```
    </div>
  );
}
export default Home;
```

Program – Students.js

```
function Students() {
  const students = [
    { id: 1, name: "Riya", course: "B.Tech" },
    { id: 2, name: "Amit", course: "BCA" },
    { id: 3, name: "Neha", course: "B.Tech" }
  ];
  return (
    <div>
      <h1>Student List</h1>
      {students.map(student => (
        <p key={student.id}>
          {student.name} - {student.course}
        </p>
      ))}
    </div>
  );
}
export default Students;
```

Program – About.js

```
function About() {
  return (
    <div>
      <h1>About</h1>
      <p>Student Portal is developed using React JS.</p>
    </div>
  );
}
export default About;
```

Program – App.js

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "./pages/Home";
import Students from "./pages/Students";
import About from "./pages/About";
```

```
function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/students">Students</Link> |{" "}
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/students" element={<Students />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

Expected Output

The application provides Home, Students and About navigation. Selecting a navigation link displays the corresponding React component without creating separate HTML pages.

Result

A simple Single Page Web Application was developed successfully using React JS, reusable components and React Router.

Viva Questions

- Why is React Router used?
- What are BrowserRouter, Routes and Route?
- What is the difference between Link and an HTML anchor?
- Why is key used in React lists?
- How does map() help in React?
- How does Experiment 08 extend Experiment 07?

Program / Code

src/App.js

```
import { useState } from "react";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "./pages/Home";
import Monitoring from "./pages/Monitoring";
import About from "./pages/About";
import { initialStations, threshold } from "./data";
import "./App.css";

function App() {
  const [stations, setStations] = useState(initialStations);

  function increaseTemperature(id) {
    setStations(previous => previous.map(station =>
      station.id === id
        ? { ...station, temperature: Math.min(60, station.temperature + 1) }
        : station
    ));
  }

  return (
    <BrowserRouter>
      <main className="app">
        <header className="page-header">
          <p className="eyebrow">AGRICULTURE | CLIMATE INTELLIGENCE</p>
          <h1>Heatwave Monitoring Portal</h1>
          <p>Monitoring, Prediction and Early Warning</p>
        </header>
        <nav aria-label="Main navigation">
          <Link to="/">Home</Link>
          <Link to="/monitoring">Monitoring</Link>
          <Link to="/about">About</Link>
        </nav>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/monitoring" element={
            <Monitoring stations={stations} threshold={threshold}
              onIncrease={increaseTemperature}
              onReset={() => setStations(initialStations)} />
          } />
          <Route path="/about" element={<About />} />
          <Route path="*" element={
            <section className="panel"><h2>Page not found</h2>
              <Link to="/">Return to Home</Link></section>
          } />
        </Routes>
        <footer>Classroom simulation | Demo heatwave threshold: 45 °C</footer>
      </main>
    </BrowserRouter>
  );
}

export default App;
```

Program / Code

src/pages/Home.js

```
import { Link } from "react-router-dom";

function Home() {
  return (
    <section className="panel">
      <p className="section-label">HOME</p>
      <h2>Welcome to the Heatwave Monitoring Portal</h2>
      <p>A simple agriculture climate intelligence application for
        monitoring temperature and displaying early warnings.</p>
      <div className="feature-grid">
        <article><h3>Monitor</h3>
          <p>View sample readings from three weather stations.</p></article>
        <article><h3>Predict</h3>
          <p>Compare the current reading with a simulated prediction.</p>
        </article>
        <article><h3>Warn</h3>
          <p>Identify normal conditions, early warnings and heatwave alerts.</p>
        </article>
      </div>
      <Link className="button" to="/monitoring">Open Monitoring</Link>
      <p className="note">Use Home, Monitoring and About to switch views
        without reloading the whole page.</p>
    </section>
  );
}

export default Home;
```

src/pages/Monitoring.js

```
import StationCard from "../components/StationCard";

function Monitoring({ stations, threshold, onIncrease, onReset }) {
  return (
    <section className="panel">
      <p className="section-label">MONITORING</p>
      <h2>Climate Monitoring Stations</h2>
      <p>Compare temperatures, simulated predictions and warning levels.</p>
      <div className="station-grid">
        {stations.map(station => (
          <StationCard key={station.id} station={station}
            threshold={threshold} onIncrease={onIncrease} />
        ))}
      </div>
      <button className="secondary" onClick={onReset}>Reset Readings</button>
      <p className="note">Prediction = current temperature + 1 °C,
        capped at 60 °C. Readings stay unchanged when switching views.</p>
    </section>
  );
}

export default Monitoring;
```

src/data.js

```
export const threshold = 45;

export const initialStations = [
  { id: "AWS001", location: "Mumbai", crop: "Maize",
    temperature: 42, rise: 1 },
  { id: "AWS002", location: "Nashik", crop: "Wheat",
    temperature: 44, rise: 1 },
  { id: "AWS003", location: "Nagpur", crop: "Cotton",
    temperature: 46, rise: 1 }
];
```

Program / Code

src/pages/About.js

```
function About() {
  return (
    <section className="panel">
      <p className="section-label">ABOUT</p>
      <h2>About This Application</h2>
      <p>This single page application is developed using React JS and
        React Router for the agriculture climate intelligence case study.</p>
      <h3>Application features</h3>
      <ul>
        <li>Three views: Home, Monitoring and About.</li>
        <li>Client-side navigation using React Router.</li>
        <li>Reusable station cards generated using map() and unique keys.</li>
        <li>Temperature updates using React state and button events.</li>
      </ul>
      <div className="information">
        <h3>How the demonstration works</h3>
        <p>Normal: current and predicted readings are below 45 °C.</p>
        <p>Early Warning: current is below 45 °C; prediction reaches 45 °C.</p>
        <p>Heatwave Warning: current temperature is 45 °C or above.</p>
      </div>
      <p className="note">The readings and predictions are simulated.
        The 45 °C threshold is a project assumption. No live weather data
        is fetched and no external notifications are sent.</p>
    </section>
  );
}

export default About;
```

src/components/StationCard.js

```
function StationCard({ station, threshold, onIncrease }) {
  const predicted = Math.min(60, station.temperature + station.rise);
  const heatwave = station.temperature >= threshold;
  const early = !heatwave && predicted >= threshold;
  const status = heatwave ? "Heatwave Warning"
    : early ? "Early Warning" : "Normal";
  const tone = heatwave ? "warning" : early ? "watch" : "normal";

  return (
    <article className="station-card" aria-label={`${station.location} station`}>
      <p className="station-id">{station.id}</p>
      <h3>{station.location}</h3>
      <p className="crop">Crop: {station.crop}</p>
      <div className="readings" aria-live="polite">
        <p>Current temperature</p>
        <strong className="current-temperature">
          {station.temperature} °C
        </strong>
        <p>Tomorrow (simulated)</p>
        <strong className="predicted-temperature">{predicted} °C</strong>
      </div>
      <p className={`status ${tone}`} role="status">{status}</p>
      <button disabled={station.temperature >= 60}
        onClick={() => onIncrease(station.id)}>Increase +1 °C</button>
    </article>
  );
}

export default StationCard;
```

Program / Code

src/App.css

```
* { box-sizing: border-box; }
body { margin: 0; background: #f1f7f3; color: #183c2b;
  font-family: Arial, sans-serif; line-height: 1.5; }
.app { max-width: 1140px; margin: 26px auto; padding: 0 24px; }
.page-header { padding: 22px 28px; background: #216644;
  color: white; border-radius: 10px 10px 0 0; }
.page-header h1 { font-size: 29px; margin: 4px 0; }
.page-header p { margin: 0; }
.eyebrow { font-size: 12px; letter-spacing: 1.4px; font-weight: bold; }
nav { display: flex; gap: 10px; padding: 13px 22px;
  background: #e1eee5; border-radius: 0 0 10px 10px; }
nav a { color: #183c2b; text-decoration: none;
  padding: 7px 18px; border-radius: 4px; font-weight: bold; }
nav a:hover, nav a:focus-visible { background: #cbded1; }
.panel { background: white; border: 1px solid #cbded1;
  border-radius: 10px; padding: 26px; margin-top: 20px; }
.section-label { margin: 0 0 4px; font-size: 12px;
  letter-spacing: 1.2px; font-weight: bold; color: #607567; }
h2 { font-size: 24px; margin: 0 0 10px; }
h3 { font-size: 19px; margin: 0 0 8px; }
p { margin: 0 0 14px; }
.feature-grid, .station-grid { display: grid;
  grid-template-columns: repeat(3, 1fr); gap: 18px; margin: 20px 0; }
.feature-grid article { background: #edf5ef;
  border: 1px solid #cbded1; border-radius: 6px; padding: 18px; }
.feature-grid p { margin: 0; color: #516359; }
.station-card { border: 1px solid #cbded1;
  border-radius: 7px; padding: 20px; }
.station-card h3 { font-size: 23px; margin: 0; }
.station-id { margin: 0; color: #516359; font-size: 12px; }
.crop { margin: 4px 0 16px; font-size: 14px; }
.readings p { margin: 8px 0 0; color: #516359; font-size: 13px; }
.readings strong { font-size: 27px; }
.predicted-temperature { color: #607567; }
.status { margin: 16px 0; padding: 11px 9px;
  border-left: 4px solid; font-weight: bold; font-size: 14px; }
.normal { color: #215a36; background: #e7f6eb; border-color: #287440; }
.watch { color: #714600; background: #fff3d6; border-color: #af7200; }
.warning { color: #8b2020; background: #ffe9e6; border-color: #b32323; }
button, .button { display: inline-block; padding: 10px 18px; border: 0;
  border-radius: 4px; background: #216644; color: white;
  text-decoration: none; font: inherit; font-size: 14px;
  font-weight: bold; cursor: pointer; }
button:hover, .button:hover { filter: brightness(.94); }
button:focus-visible, .button:focus-visible {
  outline: 2px solid #216644; outline-offset: 3px; }
button:disabled { opacity: .5; cursor: not-allowed; }
.secondary { background: #e6efe9; color: #183c2b; }
.information { padding: 18px; background: #edf5ef; border-radius: 6px; }
.information p { margin: 5px 0; }
li { margin: 5px 0; }
.note { margin: 16px 0 0; color: #516359; font-size: 13px; }
footer { text-align: center; color: #526258; margin: 16px 0; font-size: 12px; }
@media (max-width: 760px) {
  .feature-grid, .station-grid { grid-template-columns: 1fr; }
  .app { padding: 0 12px; margin: 14px auto; }
  .page-header, .panel { padding: 20px; }
  .page-header h1 { font-size: 25px; }
  nav { padding: 10px; gap: 4px; }
  nav a { padding: 6px 12px; }
}
```

Supporting Files

src/index.js

```
import React from "react";
import { createRoot } from "react-dom/client";
import App from "./App";

const root = createRoot(document.getElementById("root"));
root.render(<React.StrictMode><App /></React.StrictMode>);
```

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <meta name="theme-color" content="#216644" />
  <title>Heatwave Monitoring Portal</title>
</head>
<body>
  <noscript>Enable JavaScript to run this React application.</noscript>
  <div id="root"></div>
</body>
</html>
```

package.json - Dependencies and Commands

The complete package.json and lock file are included in the code ZIP. The main dependencies and commands are:

```
{
  "name": "heatwave-portal",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "react": "19.3.0",
    "react-dom": "19.3.0",
    "react-router-dom": "7.9.4",
    "react-scripts": "5.0.1"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build"
  }
}
```

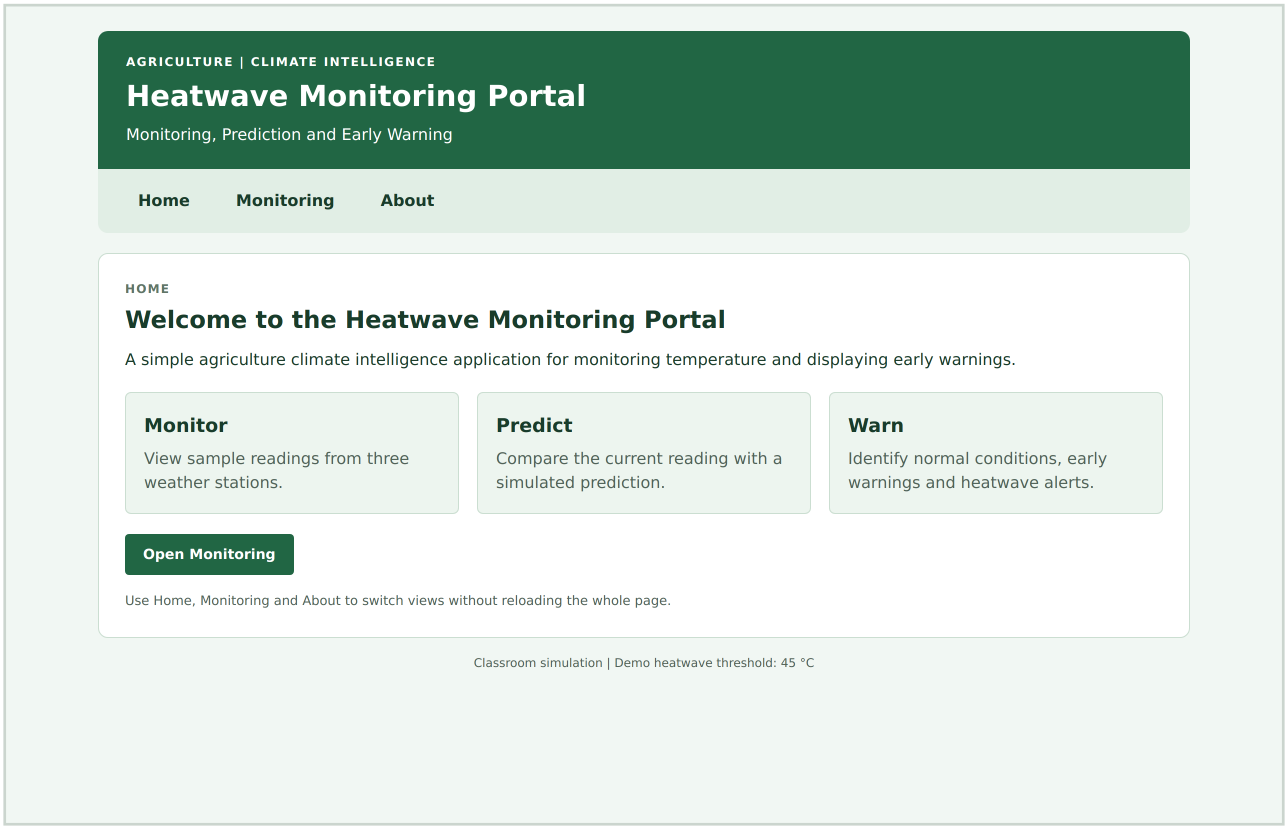
Expected Output

The browser displays a common header and Home, Monitoring and About links. Each link displays its corresponding component without reloading the whole page. Monitoring displays three cards generated from the station array. The initial temperatures are 42 °C, 44 °C and 46 °C.

Output / Screenshot

1. Home View - Route: /

The Home component displays the welcome message, application overview and Open Monitoring link.



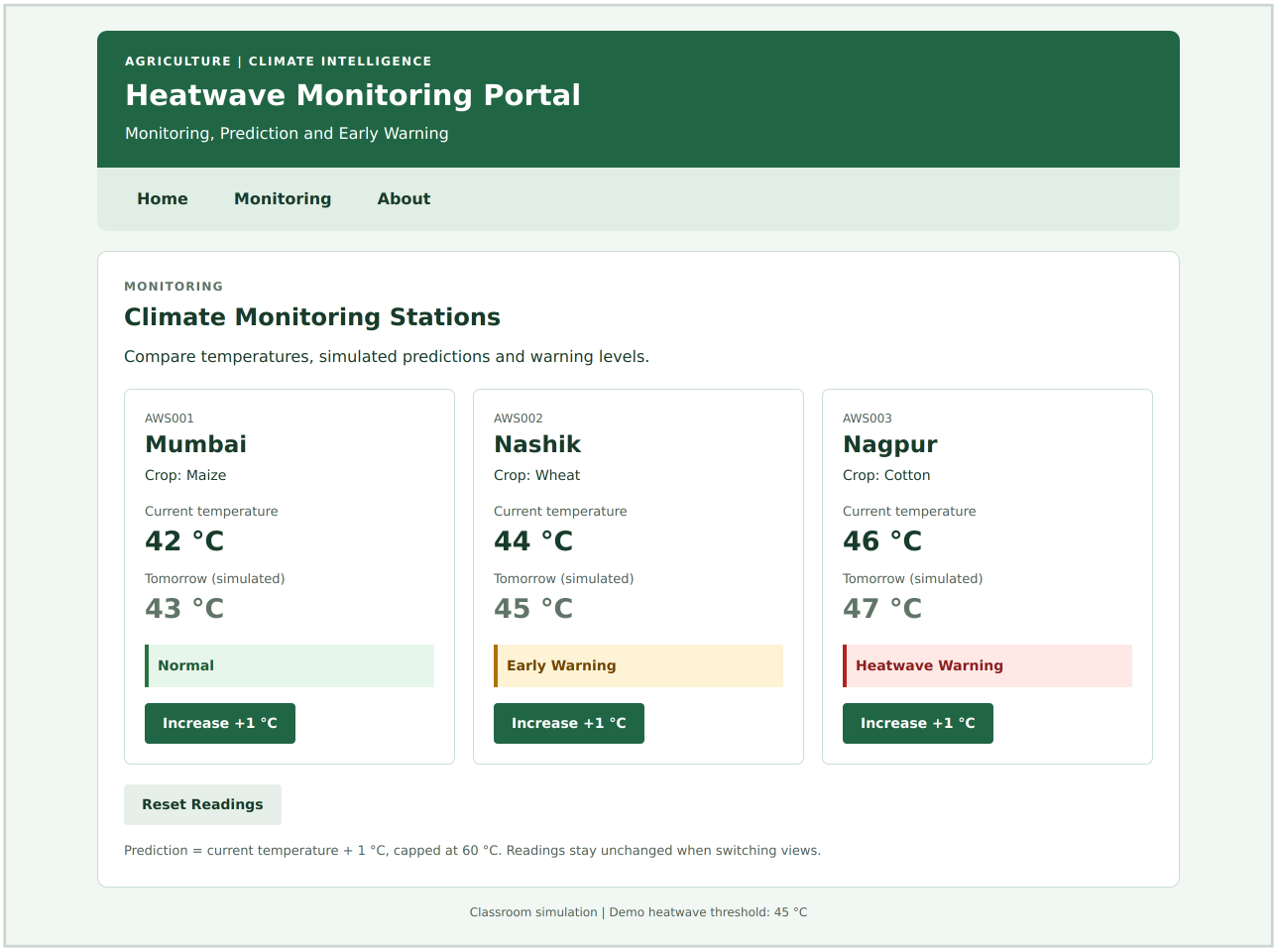
To reproduce: Run npm start and open the root URL, or select Home in the navigation bar.

Actual capture of the supplied React application. Browser viewport: 1280 × 820 CSS pixels; the PNG captures the complete page.

Output / Screenshot

2. Monitoring View - Route: /monitoring

Three cards are generated using map(). Mumbai shows 42 °C / 43 °C (Normal); Nashik shows 44 °C / 45 °C (Early Warning); Nagpur shows 46 °C / 47 °C (Heatwave Warning).



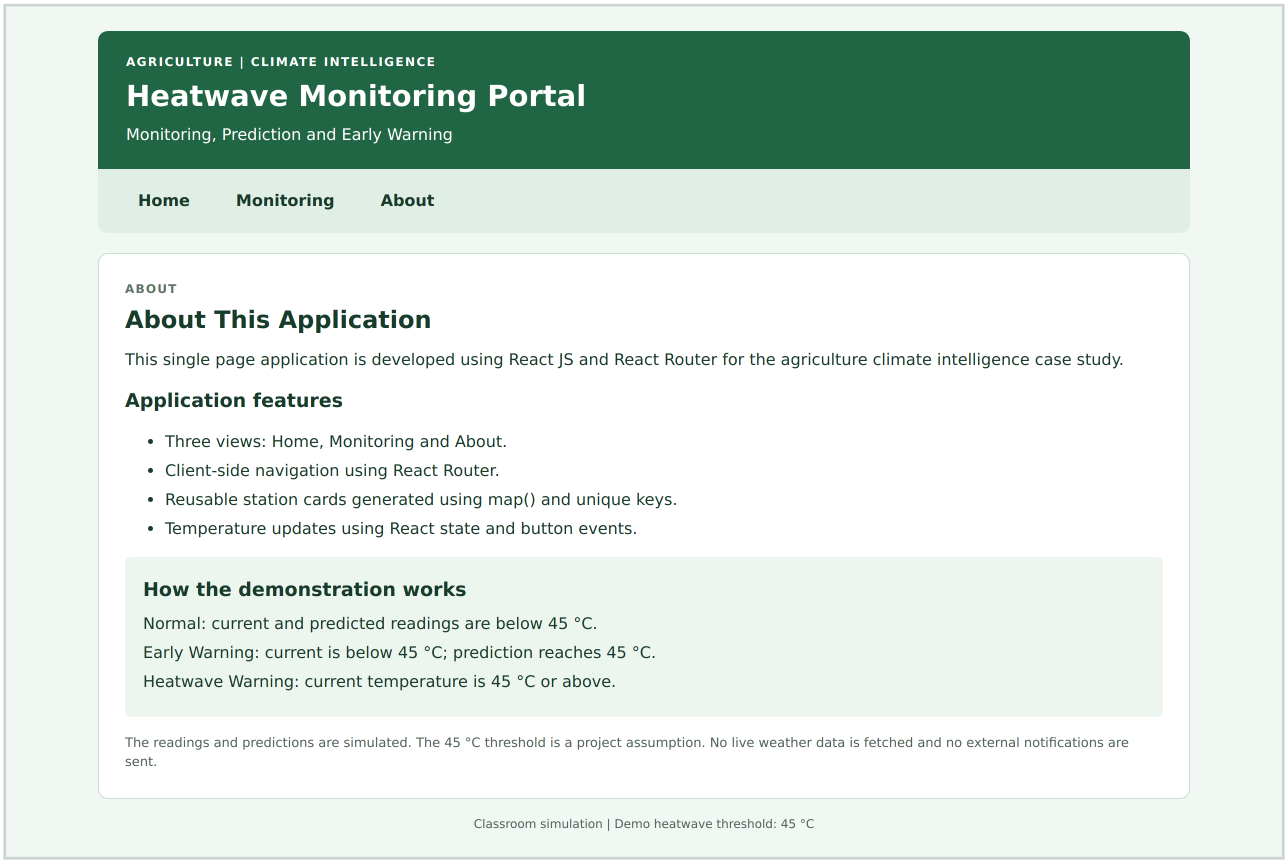
To reproduce: Select Monitoring from the navigation bar. If readings were changed, select Reset Readings before taking this output.

Actual capture of the supplied React application. Browser viewport: 1280 × 820 CSS pixels; the PNG captures the complete page.

Output / Screenshot

3. About View - Route: /about

The About component displays application information, React features and the demonstration rules.



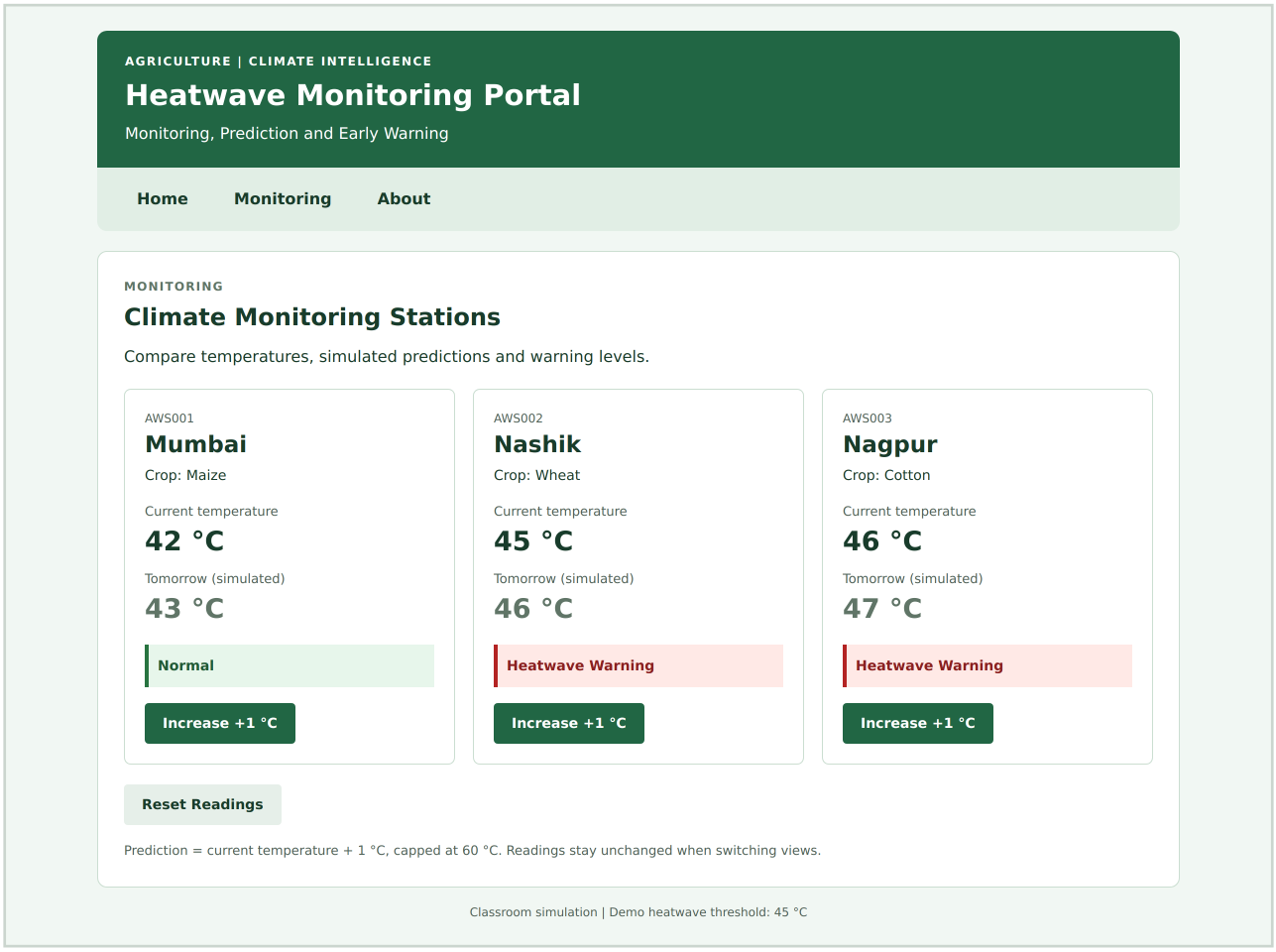
To reproduce: Select About from the navigation bar.

Actual capture of the supplied React application. Browser viewport: 1280 × 820 CSS pixels; the PNG captures the complete page.

Output / Screenshot

4. Temperature Update - State and Event Handling

Nashik changes from 44 °C to 45 °C, its prediction becomes 46 °C and its status changes from Early Warning to Heatwave Warning. Mumbai and Nagpur remain unchanged.



To reproduce: Select Monitoring, select Reset Readings, and then click Increase +1 °C on the Nashik card once.

Actual capture of the supplied React application. Browser viewport: 1280 × 820 CSS pixels; the PNG captures the complete page.

Testing and Observations

• All three routes displayed the correct component, including direct URL loads and refreshes.

• Internal Link navigation changed the route without requesting a new browser document. Back and Forward also worked.

• Monitoring displayed exactly three station cards, with no React key warnings.

• Increasing Nashik changed only its reading and warning. The updated value remained after switching views.

• Reset restored the sample readings. Increase was disabled at 60 °C; the simulated prediction was capped at 60 °C.

• The mobile layout had no horizontal overflow. The production build passed, with no browser script or console errors.

Result

The Heatwave Monitoring Portal SPA was developed successfully using React JS, reusable components, JavaScript arrays, JSX, state and React Router. Home, Monitoring and About were displayed through client-side navigation.

Conclusion and Discussion

In this experiment, I extended the React component concepts from Experiment 7 to build a single page application. I used React Router to switch views and map() to generate three reusable station cards. I also tested temperature updates, navigation and reset. This helped me understand how components, data and routes work together in a React application.

Viva Questions and Answers

1. Why is React Router used?
React Router connects URL paths to React components and provides client-side navigation between views without a full page reload.

2. What are BrowserRouter, Routes and Route?
BrowserRouter provides routing using browser history. Routes selects the matching route, and Route connects a path to the component that should be displayed.

3. What is the difference between Link and an HTML anchor?
Link with a to property performs internal React Router navigation. A normal anchor with href usually starts a browser document navigation when opening another page.

4. Why is key used in React lists?
A key gives each item a stable identity so React can match list items when data changes. This application uses the unique station.id.

5. How does map() help in React?
map() converts array items into JSX elements or components. Here it converts each station object into a StationCard.

6. How does Experiment 08 extend Experiment 07?
Experiment 7 introduced JSX, components, props, state and events. Experiment 8 adds multiple views, React Router navigation and list rendering from an array using map().